FINAL PROJECT REPORT

COLLEGE TIMETABLE GENERATOR

Conflict-Free Class Scheduling Using Graph Coloring, Greedy Strategy & Backtracking

Project Domain: College Timetable Generator

Technologies: Python, Flask, SQLite, HTML, CSS, JavaScript

Algorithms: Graph Coloring, Greedy Strategy, Backtracking

TEAM MEMBERS

G Dheeraj

G. Vishnu Vardhan Reddy

Chinna Raju

Shaik Shareef

DEPARTMENT: Artificial Intelligence and Machine Learning

COLLEGE: Malla Reddy Technical Campus

ACADEMIC YEAR: 2026–2027

1. COVER PAGE

Project Title: College Timetable Generator

Conflict-Free Class Scheduling Using Graph Coloring, Greedy Strategy & Backtracking

The College Timetable Generator is an algorithm-based web application designed to generate conflict-free college timetables. The system considers subjects, faculty, rooms, student batches, available periods, and other scheduling constraints.

The project combines Graph Coloring, Greedy Strategy, and Backtracking to create practical and understandable timetables. The project stores scheduling information and generated schedules using SQLite.

2. ABSTRACT & INTRODUCTION

2.1 Abstract

Creating a college timetable manually is a complex task because multiple resources must be coordinated simultaneously. Subjects, faculty members, classrooms, laboratories, student batches, and available time periods need to be assigned without creating conflicts.

The College Timetable Generator provides an algorithmic solution to this problem. The system models timetable conflicts as a graph in which subjects are represented as vertices and conflicts are represented as edges. Graph Coloring is then used to assign different periods to conflicting subjects.

A Greedy Strategy provides fast initial assignments, while Backtracking is used when an assignment leads to an invalid or impossible schedule. The generated timetable is validated for faculty, room, and student-batch clashes.

2.2 Introduction

Timetable scheduling is one of the important administrative activities in educational institutions. A college timetable must allocate limited resources efficiently while satisfying several constraints.

Traditional manual scheduling requires considerable time and effort. A small change in faculty availability, classroom availability, or student batches may require several other timetable entries to be changed.

The proposed system automates this process and generates a readable timetable using algorithmic scheduling techniques.

2.3 Problem Background

Colleges need to schedule:

- Subjects
- Faculty members
- Classrooms

- Laboratories
- Student batches
- Time periods
- Resource availability

The system must ensure that the same faculty member, room, or student batch is not assigned to conflicting classes.

2.4 Proposed Solution

The proposed system creates a conflict graph and applies Graph Coloring, Greedy Strategy, and Backtracking to generate a valid timetable.

System workflow: Input → Conflict Modeling → Algorithm Processing → Validation → Timetable Output

2.5 Purpose of the Project

- Automate timetable generation.
- Reduce manual scheduling effort.
- Minimize timetable conflicts.
- Efficiently use rooms and faculty.
- Provide an understandable timetable.
- Allow schedules to be reviewed and regenerated.

3. PROBLEM STATEMENT, OBJECTIVES & SCOPE

3.1 Problem Statement

Manual college timetable preparation is time-consuming, error-prone, and difficult to modify. Colleges must schedule subjects, faculty, rooms, and student batches within limited time periods while avoiding conflicts.

Therefore, there is a need for an automated timetable generation system that can efficiently handle scheduling constraints and produce conflict-free timetables.

3.2 Objectives

1. To develop an automated college timetable generation system.
2. To model scheduling conflicts using a graph.
3. To apply Graph Coloring for period assignment.
4. To use Greedy Strategy for fast scheduling.
5. To use Backtracking to recover from invalid assignments.
6. To detect faculty conflicts.
7. To detect room conflicts.
8. To detect student-batch conflicts.
9. To provide a readable timetable.

10. To make timetable generation easier to update and regenerate.

3.3 Project Scope

The project can be used for college timetable scheduling involving:

- Multiple subjects
- Multiple faculty members
- Multiple classrooms
- Multiple student batches
- Multiple time periods
- Resource availability constraints

The prototype can be extended to support larger departments and multiple campuses.

3.4 Target Users

- College administrators
- Timetable coordinators
- Department heads
- Faculty members
- Academic coordinators

4. FEATURES & FUNCTIONAL REQUIREMENTS

4.1 Main Features

1. Subject Management

The system allows scheduling information related to subjects to be entered and processed.

2. Faculty Management

Faculty information is considered while assigning subjects to periods.

3. Room Management

Rooms are assigned while checking room availability and conflicts.

4. Student Batch Management

Student batches are considered to prevent two classes from being scheduled for the same batch simultaneously.

5. Automatic Timetable Generation

The system automatically generates a timetable using the selected algorithms.

6. Conflict Detection

The generated schedule is checked for faculty, room, and student-batch clashes.

7. Regeneration

Users can inspect the generated timetable and regenerate it when required.

8. Explainable Scheduling

The system can show selected assignments and conflicting/rejected choices, making the algorithmic decisions easier to understand.

4.2 User Functionalities

- Enter scheduling information.
- Provide subjects and resources.
- Generate a timetable.
- View generated schedules.
- Check conflicts.
- Review timetable assignments.
- Regenerate the timetable when required.

4.3 CRUD Operations

Operation	Purpose
Create	Add subjects, faculty, rooms and scheduling data
Read	View stored data and generated timetable
Update	Modify scheduling information
Delete	Remove unwanted scheduling records

4.4 Authentication & Validation

For a production version, authentication can be added for administrators and timetable coordinators.

- Required fields are provided.
- Time periods are valid.
- Resources exist.
- Duplicate/conflicting assignments are detected.
- Room and faculty availability is checked.

4.5 System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Internet connection for development/deployment where required
- Minimum 500 MB available storage

Software Requirements

- Python
- Flask
- SQLite

- Web browser
- Visual Studio Code or another IDE
- HTML/CSS/JavaScript environment

5. ALGORITHMS & METHODOLOGY

5.1 System Workflow

Input → Build Model → Schedule → Validate → Output

Step 1: Input

- Subjects
- Faculty
- Rooms
- Availability

Step 2: Build Model

The system creates a conflict graph and scheduling constraints.

Step 3: Schedule

Graph Coloring and Greedy Strategy are used for initial scheduling, while Backtracking is used when necessary.

Step 4: Validate

- Faculty clashes
- Room clashes
- Student-batch clashes

Step 5: Output

The system produces a readable timetable with scheduling information.

5.2 Graph Coloring Algorithm

In the proposed approach:

- Each subject is represented as a vertex.
- A conflict between two subjects is represented as an edge.
- Different colors represent available periods/resources.

The objective is to ensure that adjacent/conflicting subjects receive different colors.

Example: If Subject A conflicts with Subject B and Subject B conflicts with Subject C, then A and B cannot receive the same period, and B and C cannot receive the same period.

5.3 Greedy Strategy

The Greedy Strategy selects the best currently feasible assignment.

- Subject with the highest conflict degree.
- Earliest valid time slot.
- Available room.
- Available faculty.

Greedy scheduling is fast and practical for generating an initial timetable.

5.4 Backtracking Algorithm

11. Selects a subject.
12. Assigns an available period.
13. Checks constraints.
14. Continues to the next subject.
15. If a conflict makes the solution impossible, it reverses the previous decision.
16. Tries another assignment.

This provides a correctness-oriented fallback for difficult scheduling cases.

5.5 Methodology

17. Requirement Analysis
18. Problem Modeling
19. Graph Construction
20. Algorithm Design
21. Backend Development
22. Database Integration
23. Frontend Development
24. Testing
25. Performance Analysis
26. Final Deployment/Presentation

6. TECHNOLOGIES & IMPLEMENTATION

6.1 Frontend Technologies

- HTML – Structure of the web application.
- CSS – Styling and layout.
- JavaScript – User interaction and dynamic operations.

6.2 Backend Technologies

- Python
- Flask

Flask handles application routing, requests, timetable generation operations, and communication between the frontend and backend.

6.3 Database

The project uses SQLite for storing scheduling-related data and generated schedules.

6.4 Tools & APIs

The system architecture uses a REST-style API + Web UI. The frontend communicates with the Flask backend to submit scheduling information and obtain generated timetable results.

6.5 Implementation Details

Frontend Module

Responsible for input forms, timetable display, user interaction, and results presentation.

Backend Module

Responsible for receiving requests, processing scheduling information, calling scheduling algorithms, and returning timetable results.

Algorithm Module

Responsible for conflict graph construction, Graph Coloring, Greedy assignment, Backtracking, and constraint validation.

Database Module

Responsible for storing scheduling information, retrieving records, storing generated schedules, and supporting future regeneration.

7. RESULTS, TESTING & PERFORMANCE

7.1 Project Results

The system generates a timetable while checking resource conflicts.

Period	Subject	Faculty	Room	Batch
09:00–10:00	DBMS	Dr. Rao	R-101	CSE-A
10:00–11:00	DAA	Ms. Priya	R-203	CSE-B
11:00–12:00	OS	Mr. Kumar	Lab-1	CSE-A
12:00–01:00	Web Tech	Dr. Anu	R-102	CSE-B

7.2 Conflict Testing

- ✓ No faculty clash
- ✓ No room clash
- ✓ No student-batch clash

7.3 Test Cases

Test Case	Input/Condition	Expected Result
TC01	Valid subject information	Subject accepted
TC02	Same faculty in same period	Faculty conflict detected
TC03	Same room in same period	Room conflict detected

TC04	Same batch in same period	Batch conflict detected
TC05	Valid resources available	Timetable generated
TC06	No valid immediate assignment	Backtracking performed
TC07	Multiple subjects	Conflict-free assignments generated
TC08	Timetable regeneration	New valid timetable produced

7.4 Testing Methods

Functional Testing

Checks whether each feature performs its expected operation.

Integration Testing

Checks communication between frontend, Flask backend, database, and algorithm modules.

Algorithm Testing

Checks whether Graph Coloring, Greedy Strategy, and Backtracking produce valid assignments.

Constraint Testing

Checks faculty, room, and student-batch conflicts.

Performance Testing

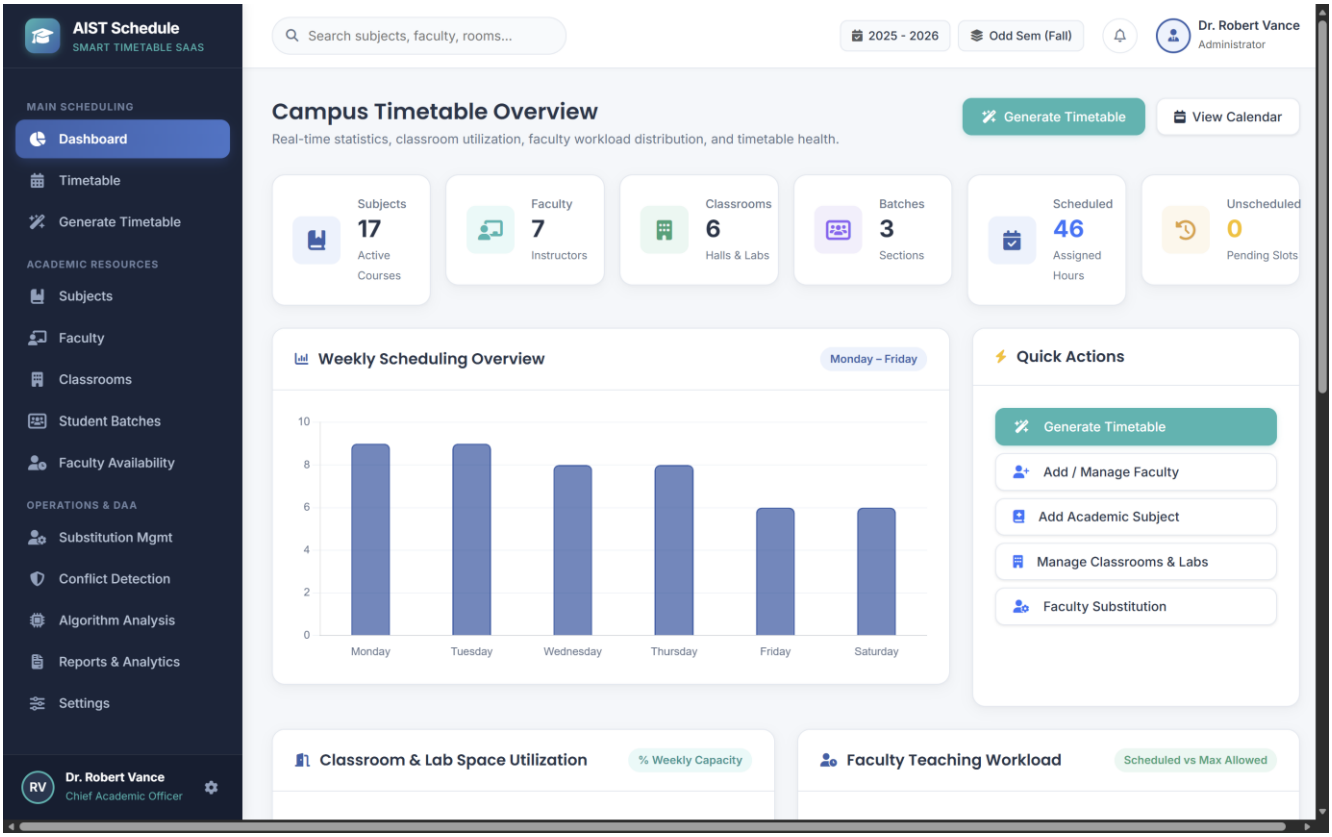
Measures timetable generation time for different numbers of subjects and constraints.

7.5 Performance Analysis

The Greedy Strategy provides fast initial assignments for normal scheduling cases. Backtracking is used only when difficult constraint combinations require recovery.

Graph Coloring is generally computationally difficult, while Greedy approaches are fast and Backtracking can grow exponentially in the worst case. Therefore, combining the three techniques provides a practical balance between speed and correctness.

Screenshots/output



7.6 Practical Benefits

- Less manual effort
- Fewer scheduling errors
- Easier timetable updates
- Transparent algorithmic decisions
- Conflict-free timetable generation

8. CONCLUSION, FUTURE SCOPE & REFERENCES

8.1 Conclusion

The College Timetable Generator provides an algorithm-centric solution to the complex problem of college timetable scheduling.

The system models scheduling conflicts as a graph and combines Graph Coloring, Greedy Strategy, and Backtracking to generate valid schedules.

Graph Coloring provides the conflict model, Greedy Strategy provides fast initial assignments, and Backtracking helps recover from difficult constraint combinations.

The resulting system reduces manual effort and provides a timetable that is easy to read, verify, and update. The project demonstrates how Design and Analysis of Algorithms concepts can be applied to a practical real-world problem.

8.2 Future Scope

AI-Assisted Scheduling

AI can be used to consider scheduling preferences and improve timetable quality.

Faculty Workload Balancing

The system can distribute teaching workloads more effectively among faculty.

Automatic Rescheduling

If a faculty member or room becomes unavailable, the system can automatically generate an updated timetable.

Analytics

Future versions can provide statistics and analytics related to room utilization, faculty workload, period utilization, and scheduling conflicts.

Scalable Deployment

The system can be extended through containerized deployment, caching, role-based access, monitoring, multi-department support, and multi-campus support.

8.3 References / Documentation

27. Project presentation: College Timetable Generator – Hackathon Project.
28. Python Documentation – Python programming language.
29. Flask Documentation – Web application development framework.
30. SQLite Documentation – Lightweight relational database.
31. HTML, CSS and JavaScript documentation.
32. Design and Analysis of Algorithms – Graph Coloring, Greedy Algorithms and Backtracking.
33. Project source code and implementation documentation.

PROJECT SUMMARY

The College Timetable Generator transforms a complex scheduling problem into an explainable algorithmic workflow:

Subjects + Faculty + Rooms + Batches + Availability

↓

Conflict Graph

↓

Graph Coloring

↓

Greedy Assignment

↓

Backtracking when required

↓

Conflict Validation

↓

CONFLICT-FREE COLLEGE TIMETABLE

This project demonstrates the practical application of DAA algorithms with Python, Flask, SQLite, HTML, CSS, and JavaScript to solve a real-world college scheduling problem.